Recurrent Neural Networks for NLP

CS 115B - Spring 2026

James Pustejovsky

March 12, 2026

Neural NLP Architecture Progression

So far:

- CNNs: Local patterns, parallel processing
- Good for: Classification, n-gram detection
- Limitation: Fixed receptive field

Today: RNNs

- Process sequences recursively
- Maintain hidden state (memory)
- Handle variable-length inputs/outputs
- Natural for language

Next time: Attention

- Selective focus on input
- Addresses RNN limitations
- Foundation for Transformers

Roadmap

Part I: Basic RNNs (25 min)

- Motivation and architecture
- Forward pass and unrolling
- Backpropagation through time (BPTT)
- Vanishing/exploding gradients

Part II: Advanced Architectures (30 min)

- LSTMs: Long Short-Term Memory
- GRUs: Gated Recurrent Units
- Bidirectional RNNs

Part III: NLP Applications (25 min)

- Language modeling
- Sequence labeling
- Sequence-to-sequence models
- Detailed examples

The Sequential Nature of Language

Language is fundamentally sequential:

- Words appear in order: "dog bit man" "man bit dog"
- Context matters: "bank" (river) vs. "bank" (financial)
- Dependencies can be long-range: "The dog that chased the cat **was** hungry"
- Sentence length is variable

CNNs have limitations:

- Fixed-size windows (filter size 3, 4, 5)
- No memory of what came before
- Hard to capture long-distance dependencies

Need: Process sequences of arbitrary length while maintaining context

Feedforward vs. Recurrent

Feedforward network:

- Fixed input size
- No temporal dynamics
- $\mathbf{y} = f(\mathbf{x}; \theta)$

Recurrent network:

- Process sequences element by element
- Maintain hidden state across time
- $\mathbf{h}_t = f(\mathbf{x}_t, \mathbf{h}_{t-1}; \theta)$
- Share parameters across all time steps

Key insight: Hidden state $\mathbf{h}_t$ acts as memory

- Summarizes past inputs
- Influences future computations
- Updated at each time step

Simple RNN: The Elman Network

At each time step t:

Input: $\mathbf{x}_t \in \mathbb{R}^d$ (e.g., word embedding)
Hidden state: $\mathbf{h}_t \in \mathbb{R}^h$
Output: $\mathbf{y}_t \in \mathbb{R}^k$ (e.g., next word probabilities)

Recurrence relation:

$$\mathbf{h}_t = \tanh(\mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{W}_{xh}\mathbf{x}_t + \mathbf{b}_h)$$

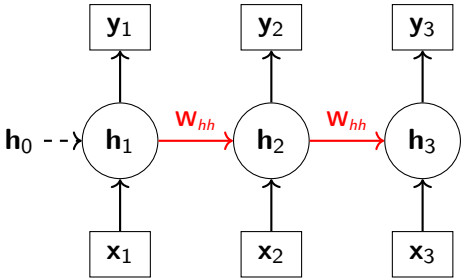
Output:

$$\mathbf{y}_t = \mathbf{W}_{hy}\mathbf{h}_t + \mathbf{b}_y$$

Parameters (shared across time):

- $\mathbf{W}_{xh} \in \mathbb{R}^{h \times d}$: Input to hidden
- $\mathbf{W}_{hh} \in \mathbb{R}^{h \times h}$: Hidden to hidden (recurrent)
- $\mathbf{W}_{hy} \in \mathbb{R}^{k \times h}$: Hidden to output
- $\mathbf{b}_h, \mathbf{b}_y$: Bias terms

RNN Computation Graph



Key observations:

- Same weights $\mathbf{W}_{hh}$ used at every time step (parameter sharing)
- Hidden state $\mathbf{h}_t$ depends on current input and previous state
- Can process sequences of any length
- Initial state $\mathbf{h}_0$ typically initialized to zeros

Example: Character-Level Language Model

Task: Predict next character given previous characters

Input sequence: "hell"
Target output: "ello"

Step-by-step:

1. $\mathbf{x}_1 = \text{embedding}(\text{"h"}) \rightarrow \mathbf{h}_1 = \text{RNN}(\mathbf{x}_1, \mathbf{h}_0) \rightarrow$ predict "e"
2. $\mathbf{x}_2 = \text{embedding}(\text{"e"}) \rightarrow \mathbf{h}_2 = \text{RNN}(\mathbf{x}_2, \mathbf{h}_1) \rightarrow$ predict "l"
3. $\mathbf{x}_3 = \text{embedding}(\text{"l"}) \rightarrow \mathbf{h}_3 = \text{RNN}(\mathbf{x}_3, \mathbf{h}_2) \rightarrow$ predict "l"
4. $\mathbf{x}_4 = \text{embedding}(\text{"l"}) \rightarrow \mathbf{h}_4 = \text{RNN}(\mathbf{x}_4, \mathbf{h}_3) \rightarrow$ predict "o"

Key: Each hidden state summarizes "history so far"

- $\mathbf{h}_3$ contains information about "hel"
- Helps predict that third "l" is followed by "o"

Detailed Computation Example

Suppose:

- Vocabulary size: 26 (letters a-z)
- Embedding dimension: $d = 50$
- Hidden dimension: $h = 128$

At time $t = 2$ (input: "e"):

1. Look up embedding: $\mathbf{x}_2 \in \mathbb{R}^{50}$
2. Compute hidden state:

$$\mathbf{h}_2 = \tanh(\mathbf{W}_{hh}\mathbf{h}_1 + \mathbf{W}_{xh}\mathbf{x}_2 + \mathbf{b}_h)$$

where:

- $\mathbf{W}_{hh}\mathbf{h}_1 \in \mathbb{R}^{128}$ (contribution from previous hidden state)
- $\mathbf{W}_{xh}\mathbf{x}_2 \in \mathbb{R}^{128}$ (contribution from current input)

3. Compute output logits:

$$\mathbf{z}_2 = \mathbf{W}_{hy}\mathbf{h}_2 + \mathbf{b}_y \in \mathbb{R}^{26}$$

4. Softmax: $P(\text{next char} = c) = \frac{\exp(z_{2,c})}{\sum_{c'} \exp(z_{2,c'})}$

Many-to-Many Architectures

Different RNN configurations for different tasks:

1. **Many-to-many (same length)**: Sequence labeling
   - Input: Words in sentence
   - Output: POS tag per word
   - $T_{\text{input}} = T_{\text{output}}$
2. **Many-to-many (different length)**: Seq2seq
   - Input: English sentence
   - Output: French sentence
   - Encoder-decoder architecture
3. **Many-to-one**: Sentiment classification
   - Input: Movie review
   - Output: Single label (pos/neg)
   - Use final hidden state $\mathbf{h}_T$
4. **One-to-many**: Image captioning

Loss Function: Language Modeling

Task: Predict next token given history

Cross-entropy loss at time t:

$$\mathcal{L}_t = -\log P(x_{t+1} \mid x_1, \ldots, x_t)$$

Total loss for sequence of length T:

$$\mathcal{L} = \frac{1}{T} \sum_{t=1}^{T} \mathcal{L}_t = -\frac{1}{T} \sum_{t=1}^{T} \log P(x_{t+1} \mid x_{\leq t})$$

In practice:

- $P(x_{t+1} \mid x_{\leq t})$ comes from softmax over $\mathbf{y}_t$
- Gold target: x_{t+1} (one-hot vector or index)
- Optimize parameters: $\mathbf{W}_{xh}, \mathbf{W}_{hh}, \mathbf{W}_{hy}, \mathbf{b}_h, \mathbf{b}_y$

Backpropagation Through Time (BPTT)

Challenge: How to compute gradients when outputs depend on previous time steps?

Solution: Unroll the recurrence, then backprop

Forward pass (left to right):

$$\mathbf{h}_1 \to \mathbf{h}_2 \to \mathbf{h}_3 \to \cdots \to \mathbf{h}_T$$

Backward pass (right to left):

$$\frac{\partial \mathcal{L}}{\partial \mathbf{h}_T} \to \frac{\partial \mathcal{L}}{\partial \mathbf{h}_{T-1}} \to \cdots \to \frac{\partial \mathcal{L}}{\partial \mathbf{h}_1}$$

Key insight: Gradients flow backward through time

- Error at time T affects parameters used at all previous steps
- Need to accumulate gradients across time

BPTT: Mathematical Derivation

Gradient of loss w.r.t. hidden state at time t:

Direct contribution from output:

$$\frac{\partial \mathcal{L}_t}{\partial \mathbf{h}_t}$$

Contribution from future (via $\mathbf{h}_{t+1}$):

$$\frac{\partial \mathcal{L}}{\partial \mathbf{h}_t} = \frac{\partial \mathcal{L}_t}{\partial \mathbf{h}_t} + \frac{\partial \mathcal{L}}{\partial \mathbf{h}_{t+1}} \frac{\partial \mathbf{h}_{t+1}}{\partial \mathbf{h}_t}$$

Since $\mathbf{h}_{t+1} = \tanh(\mathbf{W}_{hh}\mathbf{h}_t + \mathbf{W}_{xh}\mathbf{x}_{t+1} + \mathbf{b}_h)$:

$$\frac{\partial \mathbf{h}_{t+1}}{\partial \mathbf{h}_t} = \text{diag}(\tanh'(\cdot))\mathbf{W}_{hh}$$

Recursive formula:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{h}_t} = \frac{\partial \mathcal{L}_t}{\partial \mathbf{h}_t} + \mathbf{W}_{hh}^T \text{diag}(\tanh'(\cdot)) \frac{\partial \mathcal{L}}{\partial \mathbf{h}_{t+1}}$$

Gradient w.r.t. Recurrent Weights

Total gradient for $\mathbf{W}_{hh}$:

Must accumulate contributions from all time steps:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}_{hh}} = \sum_{t=1}^{T} \frac{\partial \mathcal{L}}{\partial \mathbf{h}_t} \frac{\partial \mathbf{h}_t}{\partial \mathbf{W}_{hh}}$$

where:

$$\frac{\partial \mathbf{h}_t}{\partial \mathbf{W}_{hh}} = \text{diag}(\tanh'(\cdot))\mathbf{h}_{t-1}^T$$

Similarly for other parameters:

- $\frac{\partial \mathcal{L}}{\partial \mathbf{W}_{xh}} = \sum_{t=1}^{T} \frac{\partial \mathcal{L}}{\partial \mathbf{h}_t} \text{diag}(\tanh'(\cdot))\mathbf{x}_t^T$
- $\frac{\partial \mathcal{L}}{\partial \mathbf{W}_{hy}} = \sum_{t=1}^{T} \frac{\partial \mathcal{L}_t}{\partial \mathbf{y}_t} \mathbf{h}_t^T$

Parameter sharing means gradients accumulate across all time steps!

Truncated BPTT

Problem: For long sequences, BPTT is expensive

- Memory: Must store all hidden states
- Computation: Gradients propagate through entire sequence

Solution: Truncate backpropagation after k steps

Algorithm:

1. Forward pass: Process entire sequence as normal
2. Backward pass: Only backprop k steps into the past
3. Next batch: Continue from last hidden state (detached)

Trade-off:

- Constant memory and computation
- Cannot learn dependencies longer than k steps

Common: $k = 20$ *to* 50

Long-Range Dependencies

Consider: "The cat, which was very hungry and had been wandering in the rain all day, **was** tired"

To predict "was" (singular), must remember "cat" (singular subject) from 14 words earlier

Question: Can RNNs learn such dependencies?

Problem: Vanishing gradients

- Gradients must propagate backward through many time steps
- Each step involves multiplication by $\mathbf{W}_{hh}$ and $\tanh'$
- Gradients shrink exponentially with distance

Vanishing Gradient: Mathematical Analysis

Gradient of loss at time T w.r.t. hidden state at time t:

$$\frac{\partial \mathcal{L}_T}{\partial \mathbf{h}_t} = \frac{\partial \mathcal{L}_T}{\partial \mathbf{h}_T} \prod_{k=t+1}^{T} \frac{\partial \mathbf{h}_k}{\partial \mathbf{h}_{k-1}}$$

$$= \frac{\partial \mathcal{L}_T}{\partial \mathbf{h}_T} \prod_{k=t+1}^{T} \mathbf{W}_{hh}^T \text{diag}(\tanh'(\mathbf{z}_k))$$

Key observations:

- Product of $T - t$ Jacobian matrices
- Each Jacobian: $\mathbf{W}_{hh}^T \text{diag}(\tanh'(\cdot))$
- $\tanh'(z) \leq 1$, often $\ll 1$ for large $|z|$

Result: If $||\mathbf{W}_{hh}^T \text{diag}(\tanh'(\cdot))|| < 1$:

$$\frac{\partial \mathcal{L}_T}{\partial \mathbf{h}_t} \to \mathbf{0} \text{ as } T - t \to \infty$$

Exploding Gradients

Opposite problem: If $||\mathbf{W}_{hh}|| > 1$:

$$||\frac{\partial \mathcal{L}_T}{\partial \mathbf{h}_t}|| \approx \gamma^{T-t} \to \infty$$

Consequences:

- Gradients become extremely large
- Parameter updates overshoot
- Training becomes unstable (NaN values)

Solution: Gradient clipping

$$\mathbf{g} \leftarrow \begin{cases} \mathbf{g} & \text{if } ||\mathbf{g}|| \leq \theta \\ \frac{\theta}{||\mathbf{g}||}\mathbf{g} & \text{otherwise} \end{cases}$$

Typical threshold: $\theta = 5$ or 10

Exploding gradients are easier to fix than vanishing gradients!

Why Vanishing Gradients Matter

Practical consequences:

1. **Cannot learn long-term dependencies**
   - Gradient signal from distant past is too weak
   - RNN "forgets" information after 10 steps
   - Subject-verb agreement: "The key to the cabinets **is**/**are** on the table"
2. **Slow learning**
   - Early layers receive negligible gradients
   - Parameters don't update effectively
3. **Mode collapse**
   - Network learns to ignore distant context
   - Relies only on recent words

This motivated development of LSTMs and GRUs!

LSTM: Motivation

Goal: Maintain long-term dependencies without vanishing gradients

Key innovation (Hochreiter & Schmidhuber 1997):

- Introduce **cell state** $\mathbf{c}_t$: "memory highway"
- Use **gates** to control information flow
- Additive updates to cell state (not multiplicative!)

Three gates:

1. **Forget gate** $\mathbf{f}_t$: What to remove from memory
2. **Input gate** $\mathbf{i}_t$: What new information to store
3. **Output gate** $\mathbf{o}_t$: What to output from memory

Gates are learned functions that output values in $[0, 1]$

LSTM Equations

Gate computations (all use sigmoid σ):

$$\mathbf{f}_t = \sigma(\mathbf{W}_f[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_f) \quad \text{(forget)}$$
$$\mathbf{i}_t = \sigma(\mathbf{W}_i[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_i) \quad \text{(input)}$$
$$\mathbf{o}_t = \sigma(\mathbf{W}_o[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_o) \quad \text{(output)}$$

Cell candidate (uses tanh):

$$\tilde{\mathbf{c}}_t = \tanh(\mathbf{W}_c[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_c)$$

Cell state update (element-wise operations):

$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t$$

Hidden state (output):

$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t)$$

Notation: $[\mathbf{h}_{t-1}, \mathbf{x}_t]$ is concatenation, $\odot$ is element-wise multiplication

LSTM: Intuitive Understanding

Cell state $\mathbf{c}_t$: The "conveyor belt"

- Information flows with minimal transformation
- Enables gradient flow over long sequences

Forget gate $\mathbf{f}_t$: "What to erase?"

- $f_t = 0$: Completely forget $\mathbf{c}_{t-1}$
- $f_t = 1$: Fully retain $\mathbf{c}_{t-1}$
- Example: Forget previous subject when new sentence starts

Input gate $\mathbf{i}_t$: "What new information to add?"

- Controls how much of $\tilde{\mathbf{c}}_t$ enters cell state
- Example: Remember new subject when encountered

Output gate $\mathbf{o}_t$: "What to expose?"

- Filters cell state for current prediction
- Example: Output relevant features for current word prediction

LSTM Example: Tracking Number

Task: Subject-verb agreement
Sentence: "The cat **sits** on the mat"

What LSTM might learn:

At "cat" (singular noun):

- Input gate $\mathbf{i}_t$ is high: Store "singular" in $\mathbf{c}_t$
- Forget gate $\mathbf{f}_t$ erases previous number info

At "on the" (irrelevant words):

- Forget gate $\mathbf{f}_t \approx 1$: Keep number info in memory
- Input gate $\mathbf{i}_t \approx 0$: Don't add new info

At "sits" (verb):

- Output gate $\mathbf{o}_t$ is high: Expose number from $\mathbf{c}_t$
- Helps predict singular verb form

Why LSTMs Solve Vanishing Gradients

Gradient flow through cell state:

$$\frac{\partial \mathbf{c}_t}{\partial \mathbf{c}_{t-1}} = \mathbf{f}_t$$

Contrast with vanilla RNN:

$$\frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_{t-1}} = \mathbf{W}_{hh}^T \text{diag}(\tanh'(\cdot))$$

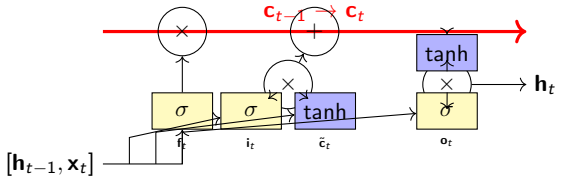
Key difference:

- LSTM: Gradient flows through *addition* and *element-wise multiplication* by forget gate
- Vanilla RNN: Gradient flows through *matrix multiplication* and tanh derivative

Result:

- If $\mathbf{f}_t \approx 1$: Gradient flows unimpeded (like residual connection)
- No repeated matrix multiplications $\rightarrow$ no exponential decay
- Can maintain gradients over hundreds of time steps

LSTM Diagram



Flow: Cell state (red) is modified by forget gate ($\times$) and input gate ($+$)

GRU: Simplified LSTM

Motivation (Cho et al. 2014):

- LSTMs work well but have many parameters
- Can we simplify while maintaining performance?

Key simplifications:

- Merge cell state and hidden state
- Two gates instead of three (update and reset)
- Fewer parameters $\rightarrow$ faster training

GRU equations:

$$\mathbf{z}_t = \sigma(\mathbf{W}_z[\mathbf{h}_{t-1}, \mathbf{x}_t]) \quad \text{(update gate)}$$

$$\mathbf{r}_t = \sigma(\mathbf{W}_r[\mathbf{h}_{t-1}, \mathbf{x}_t]) \quad \text{(reset gate)}$$

$$\tilde{\mathbf{h}}_t = \tanh(\mathbf{W}_h[\mathbf{r}_t \odot \mathbf{h}_{t-1}, \mathbf{x}_t]) \quad \text{(candidate)}$$

$$\mathbf{h}_t = (1 - \mathbf{z}_t) \odot \mathbf{h}_{t-1} + \mathbf{z}_t \odot \tilde{\mathbf{h}}_t \quad \text{(new hidden)}$$

GRU Intuition

Update gate $\mathbf{z}_t$: "How much to update?"

- $z_t = 0$: Keep old state $\mathbf{h}_{t-1}$ (like LSTM forget gate = 1)
- $z_t = 1$: Use new candidate $\tilde{\mathbf{h}}_t$ (like LSTM input gate = 1)
- Interpolation between old and new

Reset gate $\mathbf{r}_t$: "How much past to use?"

- $r_t = 0$: Ignore previous hidden state when computing candidate
- $r_t = 1$: Use full previous hidden state
- Allows model to drop irrelevant history

Final update: Linear interpolation

$$\mathbf{h}_t = (1 - \mathbf{z}_t) \odot \mathbf{h}_{t-1} + \mathbf{z}_t \odot \tilde{\mathbf{h}}_t$$

If $z_t = 1$ everywhere: full update. If $z_t = 0$: no change.

LSTM vs. GRU

	LSTM	**GRU**
Gates	3 (forget, input, output)	2 (update, reset)
State	Separate $\mathbf{c}_t$ and $\mathbf{h}_t$	Single $\mathbf{h}_t$
Parameters	More	Fewer (25% less)
Gradient flow	Through cell state	Through hidden state
Performance	Slight edge on some tasks	Comparable
Training speed	Slower	Faster

In practice:

- Both work well for most NLP tasks
- LSTM: Slight advantage on tasks requiring long memory
- GRU: Faster, simpler, good default choice
- Try both and see what works best

Motivation: Context from Both Directions

Problem with standard RNNs:

- Only see past context (left-to-right)
- Cannot see future words

Example: "I arrived at the bank after crossing the _____"

To disambiguate "bank":

- Need to see "river" (if it appears later)
- Or "street" (then financial institution)

Solution: Bidirectional RNN (Schuster & Paliwal 1997)

- Process sequence both left-to-right AND right-to-left
- Combine information from both directions
- Get full context for each position

Bidirectional RNN Architecture

Two separate RNNs:

Forward RNN ($\vec{\mathbf{h}}_t$):

$$\vec{\mathbf{h}}_t = \text{RNN}_{\text{fwd}}(\mathbf{x}_t, \vec{\mathbf{h}}_{t-1})$$

Backward RNN ($\mathbf{h}_t$):

$$\mathbf{h}_t = \text{RNN}_{\text{bwd}}(\mathbf{x}_t, \mathbf{h}_{t+1})$$

Combine (typically concatenation):

$$\mathbf{h}_t = [\vec{\mathbf{h}}_t; \mathbf{h}_t]$$

Key properties:

- $\vec{\mathbf{h}}_t$ has information from $x_1, \ldots, x_t$ (past)
- $\mathbf{h}_t$ has information from $x_t, \ldots, x_T$ (future)
- $\mathbf{h}_t$ has full sentence context

BiLSTM for POS Tagging

Example: "The cat sat on the mat"

Architecture:

1. Embeddings: Convert words to vectors
2. Forward LSTM: Process left-to-right $\rightarrow \vec{\mathbf{h}}_1, \ldots, \vec{\mathbf{h}}_6$
3. Backward LSTM: Process right-to-left $\rightarrow \mathbf{h}_1, \ldots, \mathbf{h}_6$
4. Concatenate: $\mathbf{h}_t = [\vec{\mathbf{h}}_t; \mathbf{h}_t]$
5. Linear + softmax: Predict POS tag for each word

At position 3 ("sat"):

- $\vec{\mathbf{h}}_3$ knows "The cat" came before
- $\mathbf{h}_3$ knows "on the mat" comes after
- Combined: Can better disambiguate as VERB (not NOUN)

BiLSTMs are standard for sequence labeling tasks!

Language Modeling

Task: Compute $P(x_1, x_2, \ldots, x_T)$

Factorization:

$$P(x_1, \ldots, x_T) = \prod_{t=1}^{T} P(x_t \mid x_1, \ldots, x_{t-1})$$

RNN language model:

1. At time t: Input $= x_{t-1}$, Target $= x_t$
2. Hidden state $\mathbf{h}_t$ encodes $x_1, \ldots, x_{t-1}$
3. Output: $P(x_t \mid x_{<t}) = \text{softmax}(\mathbf{W}_{hy}\mathbf{h}_t)$

Evaluation: Perplexity

$$\text{PPL} = \exp\left(-\frac{1}{T}\sum_{t=1}^{T} \log P(x_t \mid x_{<t})\right)$$

Lower perplexity = better model

Generating Text from RNN

Sampling procedure:

1. Initialize with start token: $x_1 = \langle \text{START} \rangle$
2. For $t = 1, 2, 3, \ldots$:
   - Compute $\mathbf{h}_t = \text{RNN}(x_t, \mathbf{h}_{t-1})$
   - Get distribution: $P(x_{t+1} \mid x_{\leq t}) = \text{softmax}(\mathbf{W}_{hy}\mathbf{h}_t)$
   - Sample: $x_{t+1} \sim P(x_{t+1} \mid x_{\leq t})$
3. Stop when $x_{t+1} = \langle \text{END} \rangle$ or max length reached

Sampling strategies:

- **Greedy**: $x_{t+1} = \arg\max P(x_{t+1} \mid x_{\leq t})$
- **Random sampling**: Sample from full distribution
- **Top-k sampling**: Sample from top k most likely words
- **Temperature**: $P_T(x) = \frac{\exp(z/T)}{\sum \exp(z'/T)}$ (T ¿ 1: more random)

Sequence-to-Sequence (Seq2Seq)

Architecture (Sutskever et al. 2014):

Encoder: Process input sequence $\rightarrow$ fixed-size context vector

- Input: English sentence
- RNN produces hidden states $\mathbf{h}_1, \ldots, \mathbf{h}_T$
- Context: $\mathbf{c} = \mathbf{h}_T$ (final hidden state)

Decoder: Generate output sequence from context

- Initialize with context: $\mathbf{s}_0 = \mathbf{c}$
- At each step: $\mathbf{s}_t = \text{RNN}(y_{t-1}, \mathbf{s}_{t-1})$
- Output: $P(y_t \mid y_{<t}, \mathbf{c}) = \text{softmax}(\mathbf{W}_s \mathbf{s}_t)$

Training: Teacher forcing

- Use gold target y_{t-1} as input (not model's prediction)
- Speeds up training, prevents error accumulation

Seq2Seq Example: Machine Translation

Input: "The cat sat on the mat"
Output: "Le chat était assis sur le tapis"

Encoder:

- Process English words left-to-right
- $\mathbf{h}_1$ (The), $\mathbf{h}_2$ (cat), ..., $\mathbf{h}_6$ (mat)
- Final state $\mathbf{h}_6$ = sentence representation

Decoder:

- Initialize: $\mathbf{s}_0 = \mathbf{h}_6$
- Generate: $\langle \text{START} \rangle \rightarrow$ "Le" $\rightarrow$ "chat" $\rightarrow$ "était" $\rightarrow$ ...
- Each word conditioned on context + previous French words

Challenge: Everything compressed into single vector $\mathbf{h}_6$!

This limitation motivated attention mechanisms (next lecture)

Named Entity Recognition with BiLSTM

Task: Label each word with entity type

Example: "Barack Obama visited Microsoft"
Labels: B-PER I-PER O B-ORG

Architecture:

1. Word embeddings: 300-dim vectors
2. BiLSTM: Forward and backward, hidden dim = 256
3. Concatenate: $\mathbf{h}_t = [\vec{\mathbf{h}}_t; \mathbf{h}_t] \in \mathbb{R}^{512}$
4. Linear + softmax: 512 $\rightarrow$ 9 classes (B-/I-/O for PER, ORG, LOC, MISC)
5. Optional: CRF layer on top for label dependencies

Why BiLSTM helps:

- "Obama" benefits from seeing both "Barack" (before) and "visited" (after)
- Disambiguates: "Washington" (person vs. location) based on context

Training Tips

1. **Gradient clipping**:
   - Essential for RNNs
   - Clip by norm: threshold = 5-10
2. **Initialization**:
   - Recurrent weights: Orthogonal initialization
   - Input weights: Xavier/Glorot
   - Biases: Usually zero (LSTM forget bias $\rightarrow$ 1)
3. **Optimization**:
   - Adam works well
   - Learning rate: 0.001-0.01 (tune)
   - Learning rate decay: Reduce by 0.5-0.8 when plateaus
4. **Regularization**:
   - Dropout: On inputs and outputs, NOT on recurrent connections
   - Variational dropout: Same mask across time steps

PyTorch Implementation

```
import torch.nn as nn

class LSTM_LM(nn.Module):
    def __init__(self, vocab_size, embed_dim, hidden_dim,
                 num_layers=2):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, embed_dim)
        self.lstm = nn.LSTM(embed_dim, hidden_dim,
                            num_layers,
                            batch_first=True,
                            dropout=0.3)
        self.fc = nn.Linear(hidden_dim, vocab_size)

    def forward(self, x, hidden=None):
        # x: (batch, seq_len)
        embedded = self.embedding(x)  # (batch, seq_len, embed_dim)
        output, hidden = self.lstm(embedded, hidden)
        # output: (batch, seq_len, hidden_dim)
```

Common Pitfalls

1. **Forgetting to detach hidden state**:
   - Between batches: `hidden = hidden.detach()`
   - Prevents backprop through entire training set
2. **Not shuffling for classification**:
   - Unlike language modeling, shuffle batches for sentiment/NER
3. **Using wrong packed sequences**:
   - Variable length sequences need `pack_padded_sequence`
   - Improves efficiency, prevents processing padding
4. **Incorrect tensor dimensions**:
   - Check: (batch, seq, features) vs (seq, batch, features)
   - Set `batch_first=True` for clarity
5. **Exploding gradients without clipping**:
   - Always use: `torch.nn.utils.clip_grad_norm_`

Key Takeaways

RNNs:

- Process sequences recursively with shared parameters
- Hidden state acts as memory
- Trained via backpropagation through time (BPTT)

Challenges:

- Vanishing gradients prevent learning long dependencies
- Exploding gradients (solved with clipping)
- Sequential processing (not parallelizable)

Solutions:

- LSTMs: Gates + cell state for long-term memory
- GRUs: Simpler variant with comparable performance
- BiLSTMs: Full context from both directions

Applications:

- Language modeling, sequence labeling, seq2seq

RNNs in Modern NLP

2015-2017: RNNs dominated NLP

- State-of-the-art on most tasks
- BiLSTM-CRF for sequence labeling
- Seq2seq for machine translation

2017+: Attention and Transformers

- Transformers replace RNNs for most tasks
- Parallelizable, better long-range dependencies
- But: Built on RNN insights!

RNNs still used:

- Streaming applications (online processing)
- Low-resource settings
- Research baselines
- Understanding modern architectures

Next Time: Attention Mechanisms

Limitations of RNNs we'll address:

- Bottleneck: Single context vector for entire sequence
- Alignment: Hard to map input to output positions
- Long sequences: Information loss even with LSTMs

Attention solves these:

- Weighted combination of all encoder hidden states
- Learns where to "look" in input
- Different attention for each output position

Topics next lecture:

- Bahdanau attention (additive)
- Luong attention (multiplicative)
- Self-attention
- Bridge to Transformers

Resources

Foundational papers:

- Hochreiter & Schmidhuber (1997): "Long Short-Term Memory"
- Cho et al. (2014): "Learning Phrase Representations using RNN Encoder-Decoder"
- Sutskever et al. (2014): "Sequence to Sequence Learning with Neural Networks"

Tutorials:

- Goodfellow et al., Deep Learning, Chapter 10
- Stanford CS224N lecture notes
- Olah's blog: "Understanding LSTMs"

Implementations:

- PyTorch: torch.nn.LSTM, torch.nn.GRU
- The Annotated Encoder-Decoder

Questions?